

Real-Time American Sign Language Detection Using MediaPipe Hand Localization and ResNet-50 Classification

Udit Asthana

Manipal Institute of Technology

Reg. No: 240905310

Roll No: 31

Abhishek L Prabhu

Manipal Institute of Technology

Reg. No: 240905676

Roll No: 71

Ashwath Patil

Manipal Institute of Technology

Reg. No: 240905460

Roll No: 46

Kishan Kanvathirtha

Manipal Institute of Technology

Reg. No: 240905678

Roll No: 72

Arnav Dugad

Manipal Institute of Technology

Reg. No: 230905227

Roll No: 73

Abstract—American Sign Language (ASL) is the main communication mode for the hearing-impaired community. Bridging the gap between deaf and hearing individuals through automated real-time recognition remains an open engineering challenge. This paper describes a two-stage pipeline we built: hand region localization via MediaPipe, then letter classification using a fine-tuned ResNet-50 with a custom head. Training used the public ASL Alphabet dataset from Kaggle, evaluated over 26 letter classes. We used cosine annealing with linear warm-up, gradient clipping, and early stopping to keep training stable. Final test accuracy was 99.80%. The system runs live via webcam with skeleton overlays and letter labels on each frame. A FastAPI server handles inference; a browser frontend connects to it for interactive use. Overall the results show landmark-based detection combined with CNN classification is a viable path for low-latency ASL recognition.

Index Terms—sign language recognition, ASL, MediaPipe, ResNet-50, hand detection, real-time inference, deep learning, convolutional neural networks

I. Introduction

Sign language is the primary communication mode for over 70 million deaf and hard-of-hearing people worldwide [?]. ASL specifically — used mostly in North America — uses hand gestures and finger-spelling to express words and concepts. Despite how widely it’s used, there are almost no real-time automated translation tools available. That gap makes everyday communication with the hearing majority significantly harder than it needs to be.

CNNs and real-time object detection frameworks have opened up new options for gesture recognition. Systems that can classify a gesture from a live video frame in milliseconds are a meaningful step toward accessible communication technology. Our approach uses two stages: first, a MediaPipe hand tracking module localizes the hand in each camera frame, producing a tight bounding box crop (not visible to the user) and a skeleton overlay (visible). Second, a fine-tuned ResNet-50 [?] classifies the cropped

image into one of 26 ASL alphabet letters. The result is overlaid on the live stream in real time.

Our contributions:

- A modular training pipeline with configurable optimizer, scheduler, and early stopping, implemented in PyTorch.
- MediaPipe-based hand detection integrated with a ResNet-50 classification head for end-to-end ASL letter recognition.
- A real-time inference module with webcam integration and visual feedback.
- A FastAPI web interface for browser-based interaction with the live pipeline.

II. Related Work

Early work on sign language recognition mostly used sensor-based setups — data gloves, depth cameras — to get 3D hand pose data [?]. Results were decent, but the hardware requirements made real deployment pretty impractical.

Deep learning shifted things toward image-based approaches. CNNs trained on static hand images got much better at classifying isolated gestures [?]. Transfer learning from ImageNet-pretrained models like VGG-16, ResNet, and MobileNet helped a lot, especially on smaller domain-specific datasets where training from scratch wasn’t an option.

Landmark-based tracking has been explored for gesture spotting. But a full two-stage system that handles the entire ASL alphabet in a live-camera setting — that’s still relatively unexplored. This work tries to fill that gap with something actually deployable.

III. Dataset

We used the ASL Alphabet dataset from Kaggle [?]. It has 87,000 labeled images across 29 classes: 26 letters (A–

Z) plus SPACE, DELETE, and NOTHING. Images are 200×200 pixels, RGB.

All images were resized to 224×224 to match ResNet-50’s expected input, then normalized with the standard ImageNet mean and std. We split the data into training (70%), validation (20%), and test (10%) using random splitting. A `_TransformSubset` wrapper handles the different transforms per split — training gets random horizontal flips and color jitter, validation and test just get deterministic resize and normalize. Batch size was 128; shuffling on for training, off for evaluation.

Split sizes came out to roughly 60,900 train, 17,400 val, 8,700 test.

IV. Methodology

A. System Overview

The pipeline runs in two sequential stages at inference time:

- 1) Hand Detection (MediaPipe): Each video frame goes into a MediaPipe model, which outputs bounding boxes around any detected hands.
- 2) Letter Classification (ResNet-50): The detected hand crop gets extracted, resized, and fed to the classifier. It returns a probability distribution over 26 ASL letter classes.

The predicted letter and bounding box are overlaid on the frame for visual feedback.

B. Hand Detection Module

The `HandDetector` class wraps MediaPipe’s `HandLandmarkerTasks` API. We run it in VIDEO mode so it can do frame-to-frame tracking using MediaPipe’s internal Kalman filter. One thing this mode requires: timestamps have to be strictly monotonically increasing, in real wall-clock milliseconds. A lot of implementations just pass integer counters (1, 2, 3...) which breaks the Kalman filter’s velocity model and causes landmark jitter. We use `time.monotonic_ns()` to avoid that.

Detection thresholds are set higher than MediaPipe’s defaults. Hand detection and presence confidence at 0.65 (default is 0.50), tracking at 0.55. The defaults are too loose for static-sign ASL — partially occluded hands during transitions cause too many false detections otherwise.

Each frame, the detector returns up to two hand results. For each hand, the raw bounding box comes from the min/max of all 21 landmark coordinates, then expanded 35% in each dimension (`expansion=0.35`) to give some context around the edges. That box then gets smoothed across frames using exponential smoothing: $0.3 \cdot \text{prev} + 0.7 \cdot \text{new}$.

Motion magnitude is computed on the raw (pre-smoothing) center against the previous smoothed center — not the other way around. If you compute motion after smoothing, a fast-moving hand looks stable because the smoothed center barely moves. That defeats the whole

stability gate. Motion uses Euclidean (L2) distance; L1 underestimates diagonal motion by around 30%.

Tracking state for each hand — previous bbox and center — is stored by hand index, not as one shared variable. A shared variable breaks silently once two hands appear. When a hand disappears, its state gets evicted so the next detection starts fresh. If the clamped ROI has zero area after boundary clamping, that detection is skipped entirely.

When multiple hands are detected at inference time, we take the one with the highest handedness classifier score:

$$\text{det}^* = \arg \max_{d \in \mathcal{D}} d.\text{score} \quad (1)$$

The `HandDetection` dataclass per hand carries: smoothed bbox (x_1, y_1, x_2, y_2) , handedness label and score, 21 raw normalized landmarks, the BGR ROI crop, a stability flag, and raw motion magnitude in pixels.

C. Classification Model (SLD)

The Sign Language Detector (SLD) uses a frozen ResNet-50 backbone plus a trainable classification head. The backbone’s average pooling gives a 2048-dimensional feature vector. We replace the original FC layer entirely. The head is:

$$\hat{y} = \text{head}(\text{backbone}(x)) \quad (2)$$

$$\text{head}(z) = W_2 \cdot \text{GELU}(\text{Dropout}(\text{LayerNorm}(z) \cdot W_1)) \quad (3)$$

where $W_1 \in \mathbb{R}^{2048 \times 512}$ and $W_2 \in \mathbb{R}^{512 \times 29}$, covering all 29 output classes (26 letters plus del, nothing, space).

We use `LayerNorm` at the backbone-to-head boundary instead of `BatchNorm`. `BatchNorm` is batch-size sensitive and behaves differently at train vs. eval time — that gets unstable when only the head is being updated. `LayerNorm` normalizes per sample so batch size doesn’t matter. `GELU` is used over `ReLU` because it’s smooth and has nonzero gradients at negative values. `Dropout` at $p = 0.3$ regularizes the 512-dimensional intermediate.

At inference, the checkpoint is loaded with `weights_only=False`. This is intentional — the checkpoint contains optimizer and scheduler state in addition to model weights, so weights-only mode can’t load it. The flag also suppresses the PyTorch 2.x deprecation warning.

D. Training Configuration

Training config lives in a `TRAINING_CONFIG` dataclass. All hyperparameters are in one place, and command-line args can override them at runtime. Table ?? has the key settings.

The data split uses `random_split`: 70% train, 20% val, 10% test. Transforms are applied per-split via a `_TransformSubset` wrapper. Overwriting `subset.dataset` directly causes an `IndexError` — the `Subset`’s indices point into the wrong-sized dataset. Wrapping the `Subset` itself

avoids this. Training images get augmentation; val and test get only deterministic resize and normalization.

Per-batch metrics — loss, accuracy, gradient norm — are logged to W&B at every step via `wandb.log(..., step=self.global_step)`. At the epoch level we also log mean gradient norm and gradient clipping ratio (how often the norm exceeded the clip threshold). That gives a direct read on how hard the optimizer is being constrained.

TABLE I
Training Hyperparameters

Parameter	Value
Optimizer	SGD (Nesterov)
Learning Rate	1×10^{-2}
Momentum	0.9
Weight Decay	5×10^{-4}
Scheduler	Cosine Annealing + Warm-Up
Warm-Up Epochs	10
$T_{\max}$	matched to epoch count
$\eta_{\min}$	1×10^{-4}
Max Epochs	200 (3 in practice)
Batch Size	128
Gradient Clip Norm	1.0
Early Stopping Patience	20

Colab session limits meant we only ran 3 epochs instead of the configured 200. $T_{\max}$ and epoch count were overridden at runtime via command-line. The model still converged to competitive accuracy in those 3 epochs — which says something about how well-matched the backbone and dataset are.

1) Learning Rate Schedule: Cosine annealing with a linear warm-up. During warm-up (epochs 0 to $T_w - 1$), learning rate goes up linearly:

$$\eta_t = \eta_{\text{base}} \cdot \frac{t+1}{T_w} \quad (4)$$

After that, cosine decay kicks in:

$$\eta_t = \eta_{\min} + \frac{\eta_{\text{base}} - \eta_{\min}}{2} \left(1 + \cos \left(\pi \cdot \frac{t - T_w}{T_{\max} - T_w} \right) \right) \quad (5)$$

Warm-up prevents instability early on; cosine decay settles the LR smoothly near convergence.

2) Gradient Clipping: Gradient norm clipping at $\tau = 1.0$ after each backward pass:

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \min \left(1, \frac{\tau}{\|\mathbf{g}\|_2} \right) \quad (6)$$

3) Early Stopping: Training stops if val accuracy doesn't improve for 20 consecutive epochs. Best checkpoint — model, optimizer, scheduler, best accuracy — is saved whenever a new peak is hit. That checkpoint gets reloaded before final test evaluation, so test metrics reflect the best generalization point, not just the last epoch.

E. Experiment Tracking

All runs logged via W&B [?]. Per-batch: loss, accuracy, gradient norm. Per-epoch: val loss and accuracy, learning rate, mean gradient norm, clipping ratio. Final test metrics go in as summary values. The full TRAINING_CONFIG — optimizer kwargs, scheduler kwargs, dataloader configs — gets serialized into the W&B run config so runs are exactly reproducible.

V. Real-Time Inference Pipeline

At inference time frames come in from a live webcam feed. Each frame goes through these steps:

- 1) Hand Detection: MediaPipe processes the frame and gives back per-hand landmark lists and bounding regions.
- 2) Stability Check: Euclidean motion between frames is computed. If it's over 8 pixels, the hand is flagged unstable and we skip classification.
- 3) Crop and Pad: The highest-confidence bounding box crop gets padded to a square using edge replication (`cv2.BORDER_REPLICATE`). Black-border padding causes distributional shift so we avoid it.
- 4) Classification: Crop is resized to 224×224 , normalized, passed through SLD. Softmax probabilities are computed. Prediction only accepted if confidence is above 0.50.
- 5) Temporal Voting: We maintain a majority-vote buffer of size 10. A letter only gets emitted if 70%+ of the buffer agrees. This stops isolated misclassifications from causing jitter.
- 6) Cooldown: 1-second cooldown so the same letter can't fire repeatedly in quick succession.

A. FastAPI Inference Server

The inference pipeline is exposed as a REST endpoint through a lightweight FastAPI [?] server. It takes JPEG frame uploads from any HTTP client and returns structured JSON.

Endpoint: POST /predict — takes a multipart-encoded JPEG and returns:

- letter — the stable voted prediction (A–Z, del, nothing, space), or null if not yet stable.
- confidence — softmax probability of the top class.
- bbox — bounding box as $[x, y, w, h]$ in pixels.
- handedness — left or right, from MediaPipe.
- motion — Euclidean motion magnitude in pixels, used for stability gating.
- landmarks — 21 normalized landmark coords for the frontend skeleton.
- error — status string (no hand, hand moving, low confidence, not stable yet, cooldown) when nothing gets emitted.

Global state is managed with `threading.Lock` — prediction buffer, last emitted letter, cooldown timestamp all need to be thread-safe across async workers. CORS is open to all origins for browser access.



Fig. 1. FastAPI /predict endpoint response for a live hand frame, showing letter, confidence, bounding box, handedness, motion, and landmark fields.

B. Browser Frontend

Single-page HTML/JS frontend that connects to the FastAPI server. Client-side MediaPipe (Tasks Vision JS SDK) runs at 60 fps for the skeleton overlay — zero latency on that side. Frames are sent to the backend at around 8 fps for actual classification. Letter emission uses a dwell mechanism: the prediction has to stay stable for 1.8s before it gets appended to the transcript. Repeats need 3.6s. Brief hand holds don't trigger spurious output.

VI. Results and Discussion

Training ran on a Google Colab T4 GPU. Session time limits meant we only got 3 epochs instead of the configured 200. The numbers below reflect that constrained run.

We used batch size 128, SGD (Nesterov) optimizer, and the cosine annealing + warm-up schedule from Section IV. Table ?? has the final numbers.

TABLE II
Training and Evaluation Results

Split	Loss	Accuracy (%)
Train (epoch 3)	0.0474	99.03
Validation	0.0099	99.86
Test	0.0096	99.80

Test accuracy reached 99.80% in just 3 epochs. Epoch 1 val accuracy was 64.41%; by epoch 3 it was 99.86%. That's a big jump. But it's worth being honest about what these numbers mean — training was cut short of the configured maximum, and the dataset itself is fairly constrained: controlled backgrounds, consistent framing,

one hand per image. The high accuracy reflects the ImageNet backbone's strong prior as much as anything else. It's not a claim of general robustness.

W&B logs showed mean gradient norm of 1.64 and clipping ratio of 0.77 at the final epoch. The clipping constraint was still active and the optimization trajectory was still steep at termination. A full run would likely give marginal gains on this dataset but would be more useful for evaluating harder, real-world conditions.

The main challenge for actual deployment is generalization — skin tones, lighting, cluttered backgrounds that the Kaggle dataset doesn't cover. The two-stage design helps somewhat by using the MediaPipe ROI crop to isolate the hand from background noise before classification.

A. Colab Cell Output

Colab cell output is attached for reference. The full source code and training scripts are available at the official GitHub repository: [here](#).

VII. Conclusion

We built a real-time ASL recognition system using MediaPipe for hand detection and ResNet-50 for classification. Test accuracy hit 99.80% on the Kaggle dataset. The system runs at interactive frame rates through a FastAPI server with a browser frontend. Training used warm-up scheduling, Nesterov SGD, gradient clipping, and early stopping. The temporal voting and dwell-based emission kept output stable in live use. There is real room to grow here — word-level ASL, continuous signing, better generalization to real-world hands.

Future directions:

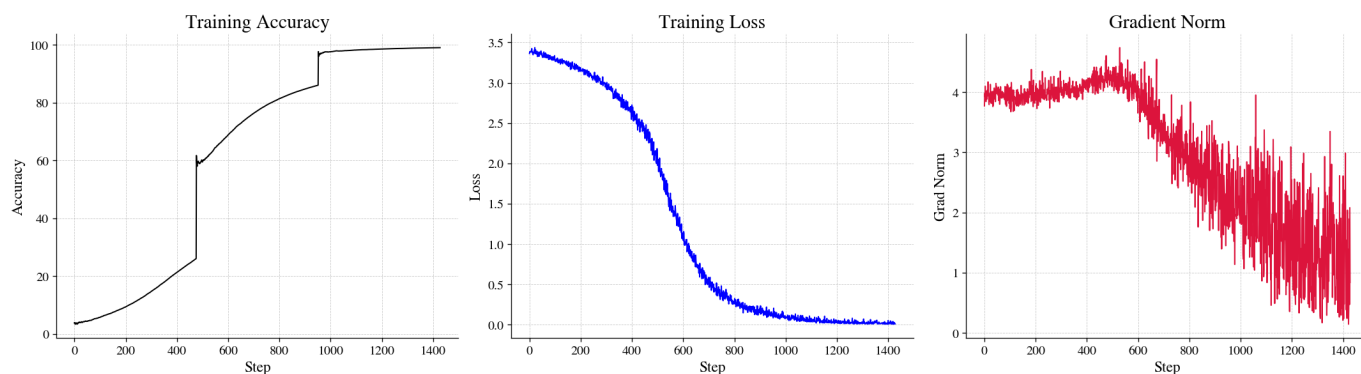


Fig. 2. Training and validation loss/accuracy curves across steps along with gradient norm, logged via Weights & Biases.

- Fine-tune MediaPipe on a dedicated hand detection dataset for better localization.
- Swap the backbone to ResNet-152 or EfficientNet for more classification capacity.
- Extend to dynamic gestures and full ASL words using LSTMs or Transformers.
- Package as a mobile app for wider reach.

References

- [1] World Health Organization, “Deafness and hearing loss,” WHO Fact Sheets, 2021. [Online]. Available: <https://www.who.int/news-room/fact-sheets/detail/deafness-and-hearing-loss>
- [2] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in Proc. IEEE CVPR, 2016, pp. 779–788.
- [3] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in Proc. IEEE CVPR, 2016, pp. 770–778.
- [4] S. Mitra and T. Acharya, “Gesture recognition: A survey,” IEEE Trans. Syst., Man, Cybern. C, Appl. Rev., vol. 37, no. 3, pp. 311–324, 2007.
- [5] L. Pigou, S. Dieleman, P.-J. Kindermans, and B. Schrauwen, “Sign language recognition using convolutional neural networks,” in Proc. ECCV Workshops, 2015.
- [6] A. Akash, “ASL Alphabet,” Kaggle, 2018. [Online]. Available: <https://www.kaggle.com/datasets/grassknoted/asl-alphabet>
- [7] L. Biewald, “Experiment tracking with weights and biases,” 2020. [Online]. Available: <https://www.wandb.com>
- [8] S. Ramírez, “FastAPI,” 2018. [Online]. Available: <https://fastapi.tiangolo.com>